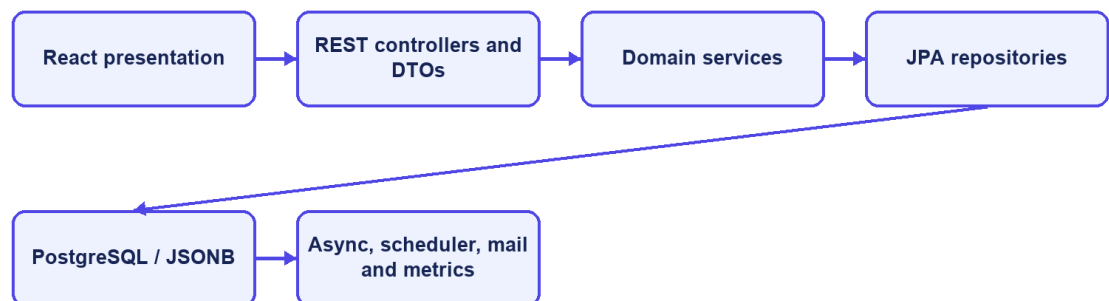


to PostgreSQL through Spring Data JPA. Cross-cutting configuration covers security, CORS, async execution, exception responses, scheduling, API documentation, and health/metrics endpoints.

The backend stores relational business state in PostgreSQL. JSONB is used for naturally variable collections or snapshots such as skills, extracted terms, vectors, reasons, benefits, policy metadata, and analytic distributions. CV binaries are stored through a storage service in a configured local path or Docker volume; the database stores file metadata and the processing result. SMTP is optional, and a no-operation mail implementation allows local workflows when outbound email is disabled.

Modular-monolith architecture



CareerFit IT AutoPilot — thesis diagram

Figure 2.5. CareerFit container and component architecture

2.12 Module Design

Table 2.2. Backend modules and responsibilities

Module	Main responsibility	Representative services
Auth and Security	Registration, password login, JWT validation, account resolution, and role rules	AuthService, JwtService, security filters
Candidate, CV, and Skill	Profile/portfolio, CV draft/review, storage, OCR cleanup, validation, and skill suggestions	CandidateProfileService, CvIngestionService, PdfExtractionService, SkillService
Job and Employer	Public catalogue, company onboarding, JD quality, draft/publish, urgent/deadline, and recruiter-owned lifecycle	JobService, EmployerService, QualityValidationService

Module	Main responsibility	Representative services
Matching	TF-IDF/cosine score, labels/reasons, skill-transfer Potential, batch matching, filters, and ranking	ScoringService, SkillTransferService, MatchingService, MatchingQueryService
Recommendation	Candidate-profile Job ordering and similar-Job retrieval	RecommendationService
Application and Talent	Apply/withdraw, applicants, invitations, status changes, Talent Pool, and CV bookmarks	ApplicationService, RecruiterTalentService
Feedback	Feedback validation, Rocchio update, and recomputation signaling	FeedbackService, RocchioService
Automation	Per-account policy, run-now, pause/resume, auto-apply, reminders, and scheduler	AutomationPolicyService, AutoApplyService, AutomationScheduler
Notification	Policy guard, delivery log, SMTP/no-op mail, and signed email actions	NotificationPolicyGuard, NotificationEmailService, EmailActionService
Analytics, Admin, and Audit	Live analytics, moderation, monitoring, audit persistence, and common API behavior	Analytics services, administrative services, audit/common components

The modules cooperate through service calls rather than direct controller-to-repository shortcuts for core workflows. Transaction boundaries belong at service operations that change a consistent group of records. Read-only queries may use dedicated query services or projections to avoid loading unrelated state. DTOs prevent persistence entities from becoming an accidental public API contract.

2.13 Data Design

2.13.1 Core Identity and Recruitment Data

`user_account` stores identity, role, activation, verification, language, and password hash. A Candidate owns a profile, multiple CVs, reviewable CV sections, extracted skills, portfolio links, and portfolio projects. A Recruiter owns an employer profile and Jobs. A Job stores its JD, structured recruitment fields, draft/publish state, urgent flag, application deadline, application count, source details, and vector data.

`matching` is the unique association between one CV and one Job. It stores the direct score, label, separate Potential flag and explanation, reasons, and recomputation state. `application` links a Candidate and Job and may reference the CV and Matching used at that time. `feedback` stores a judgment about a Matching. `recruiter_cv_bookmark` keeps a Recruiter's saved Candidate/CV for an owned Job, while `recommendation_interaction` records events such as viewed, skipped, applied, saved, not interested, or show similar.

2.13.2 Automation, Communication, and Operational Data

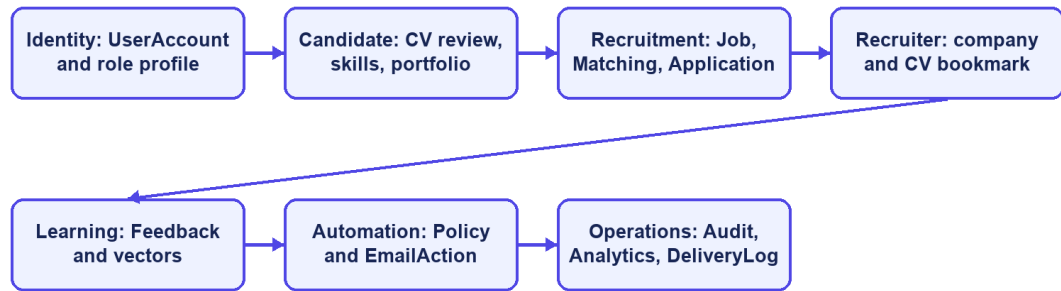
automation_policy stores one policy per account, including enable/pause state, thresholds, quotas, cooldowns, quiet hours, category guards, and notification preferences. email_action stores expiring one-time actions using a token hash. Notification delivery records support deduplication and policy decisions. audit_log records the actor, action, target, result, source channel, request details, and structured metadata. Analytics tables support aggregate dashboards without changing Matching records.

Table 2.3. Key integrity constraints

Constraint	Purpose
Unique normalized user email	Prevent duplicate identities that differ only by case
One Candidate/Recruiter profile and one policy per account	Preserve role ownership and account-level automation settings
Partial unique default CV per Candidate	Ensure at most one active default CV
Unique Matching per CV and Job	Prevent duplicate direct scores for the same pair
Unique Application per Candidate and Job	Prevent duplicate applications or invitations
Unique Recruiter bookmark per Job and Candidate	Prevent duplicate Talent Pool bookmarks
Unique feedback per Matching and actor	Keep one current judgment per actor
Check constraints on roles, states, labels, salary, and source channels	Reject invalid domain values at the database boundary
Indexes for active/urgent/popular Jobs, deadlines, ownership, and ranking	Support server-side catalogue, reminder, and matching queries

Flyway migrations are the authoritative schema history. Hibernate uses ddl-auto=validate, so the application validates entity-to-schema compatibility instead of silently generating a different database. Indexes support common access paths such as active jobs, recruiter jobs, CV ownership, matching by score, applications by candidate or job, feedback history, tokens, and audit chronology.

Data domains and relationships



CareerFit IT AutoPilot — thesis diagram

Figure 2.6. CareerFit logical entity relationships

2.14 Security, Failure, and Consistency Design

2.14.1 Authentication and Authorization

The backend is stateless at the HTTP session layer. A JWT authentication filter runs before username/password authentication processing, and a user-ID resolution filter runs after JWT validation. Public access is explicitly limited to authentication entry points, selected job/employer/analytics GET endpoints, health/metrics/API documentation, and email-action redemption. Candidate, Recruiter, and Administrator endpoint groups require corresponding roles. Automation requires authentication, with service logic responsible for policy ownership.

The security configuration disables CSRF because the application uses stateless bearer authentication, configures a CORS origin allow-list, denies framing, applies a content-security policy, and configures HSTS. These headers do not replace TLS termination in the deployed environment. Public Swagger and Prometheus access are acceptable for local development but should be restricted in a production deployment.

2.14.2 Failure Handling

CV processing represents long-running work with statuses rather than holding a browser request open until every score completes. Extraction or validation failure records a failure state and reason. CV and Job matching tasks are submitted through an after-commit executor so background work cannot read uncommitted rows. Batch scoring isolates individual pair failures, and scheduled recomputation keeps unsuccessful rows marked for later review instead of falsely marking them complete.

Email can operate through SMTP or a no-op implementation. Notification delivery outcomes are recorded as sent, skipped, or failed, and policy guards enforce deduplication, quota, cooldown, and timing rules. Email failure must not roll back an already valid matching result. Conversely, application creation and status changes use transactions and uniqueness constraints because partial persistence would change recruitment state.

2.14.3 Identified Design Risks

Table 2.4. Design risks and required treatment

Risk	Current control	Remaining treatment
Unauthorized cross-account access	URL role rules and service ownership checks	Maintain negative integration tests for every identifier-based operation
Duplicate applications or matchings	Service checks and unique constraints	Convert database conflicts into stable API errors
Replayed email action	Pending/redeemed state and expiry	Implemented hashed storage, GET confirmation, and POST execution; retain replay and expiry tests
Link scanner triggers action	GET is non-mutating and displays confirmation	Add rate limiting and deployment-specific origin controls
Scheduler/configuration drift	Scheduler timing reads from app.scheduler properties	Maintain configuration and schedule-boundary tests
Sensitive data in logs	Structured audit metadata and DTO boundaries	Review retention, redaction, access, and exported evidence
Lexical scoring bias or omission	Reasons, validation, feedback, and HITL	Evaluate representative data; never treat score as hiring probability
Local file-storage loss or exposure	Configured path and Docker volume	Define backup, encryption, malware scanning, and retention before production

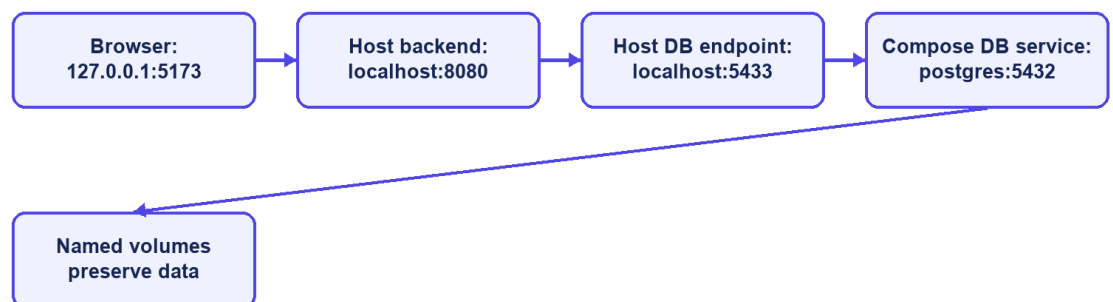
2.15 Deployment Architecture

The development deployment uses Docker Compose for PostgreSQL and optionally the backend. PostgreSQL 16 exposes container port 5432 as host port 5433 by default. A backend process running directly on the host connects to localhost:5433; the backend container connects through the Compose network to postgres:5432. Confusing these two addresses is a common configuration failure.

The optional backend container exposes port 8080, depends on PostgreSQL health, and mounts a named volume for CV storage. The React/Vite frontend is normally run separately on port 5173 during development and calls the backend API. Environment variables override database credentials, JWT secret, application base URL, CORS origins, mail settings, OCR executable and languages, and storage path. Tesseract with Vietnamese and English language data is included in the backend container path described by the project Docker configuration.

Actuator exposes health and Prometheus metrics according to the current security configuration. PostgreSQL health checks gate backend container startup. This architecture is appropriate for reproducible local development and demonstration, but it is not by itself a production topology. A production design would additionally require managed secrets, TLS termination, restricted management endpoints, database backup and recovery, protected object storage, centralized logs, monitoring alerts, and explicit scaling and retention policies.

Local deployment addresses



CareerFit IT AutoPilot — thesis diagram

Figure 2.7. Local and containerized deployment topology